

ATP Tennis Professionals Dataset

Data collected from <https://www.atptour.com/en> regarding all single matches of 10.361 male players (top 500 players who played between and 28/03/1973 and 14/02/2022)



Gestão de Big Data

Ano Letivo 2025/2026

Professor António Lopes

Professor Pedro Ramos

Grupo 4

Daniel Oliveira - 137564

Francisco Rodrigues -134018

João Maurício - 136811

Rafael Machado - 87341

Índice

1. Identificação do dataset	2
Descrição	2
2. Exploração e preparação de dados	2
Contagem de documentos	2
Estrutura dos documentos	2
Exploração e limpeza inicial de dados.....	3
Exportação dos dados para csv	9
3. Modelo relacional.....	10
Idealização do modelo relacional.....	10
Importação de dados e criação das tabelas iniciais	11
Limpeza de dados e criação de relações	13
Construção da tabela final de jogadores (tabela players)	13
Correspondência de torneios e jogadores com países	14
Ligação entre jogos, torneios e jogadores	18
Limpeza final de dados.....	19
Modelo relacional final.....	19
4. Queries SQL	20
Query 1 - Jogadores, torneios e jogos por país.....	20
Query 2 – Top 10 jogadores por winning rate	22
Query 3 – Top 10 jogadores esquerdinos por winning rate Grand Slam.....	25
Query 4 – Top 5 jogadores por winning rate em chão “Hard”	26
5. Conclusão	28

1. Identificação do dataset

Descrição

O Dataset incluído no ficheiro atpplayers.json inclui dados recolhidos de <https://www.atptour.com/en> relativos a todas as partidas individuais de 10.361 jogadores do circuito de ténis ATP (top 500 jogadores que jogaram entre 28/03/1973 e 14/02/2022)

2. Exploração e preparação de dados

Contagem de documentos

Para importar a base de dados para o MongoDB foi utilizado o seguinte comando:

- `mongoimport --username root --password root --authenticationDatabase admin --db atp --collection atpplayers --drop --file atpplayers.json`

Este comando resultou na base de dados 'atp' com a coleção 'atpplayers' e na importação de 1.308.835 documentos.

Estrutura dos documentos

Ao importar os dados para o MongoDB, analisámos a estrutura dos documentos (sendo que cada documento consiste em dados relativos a diferentes partidas de ténis), que apresentam os seguintes campos:

- `_id` – ObjectId, identificador único de cada registo
- `PlayerName` – Nome do Jogador A
- `Born` – Local de Nascimento Jogador A
- `Height` – Altura do Jogador A (em texto)
- `Hand` – Mão do Jogador A
- `LinkPlayer` – URL para a página do Jogador A no atptour.com
- `Tournament` – Nome do torneio correspondente ao jogo
- `Location` – Localização do torneio
- `Date` – Data/Intervalo de data do torneio
- `Ground` – Chão do torneio
- `Prize` – Prémio do torneio
- `GameRound` – Ronda do torneio a que a linha/jogo corresponde
- `GameRank` – Ranking do Jogador A
- `Oponent` – Nome do Jogador B (Jogador Oposto)

- WL – Resultado da Partida – (vitória/derrota...) (na perspetiva do Jogador A)
- Score – Pontos do resultado da Partida (na perspetiva do Jogador A)

```

_id: ObjectId('624ab34913b144c54b3c9abc')
PlayerName : "Novak Djokovic"
Born : "Belgrade, Serbia"
Height : 188
Hand : "Right-Handed, Two-Handed Backhand"
LinkPlayer : "https://www.atptour.com/en/players/novak-djokovic/d643/player-activity..."
Tournament : "Nitto ATP Finals"
Location : "Turin, Italy"
Date : "2021.11.15 - 2021.11.21"
Ground : "Hard"
Prize : "$7,250,000"
GameRound : "Round Robin"
GameRank : 12
Opponent : "Cameron Norrie"
WL : "W"
Score : "62 61"

```

Figura 1 - Estrutura de um documento da coleção atpplayers da base de dados atp;

Exploração e limpeza inicial de dados

Após a importação dos dados e a visualização dos campos pertencentes, o primeiro tema que explorámos consistiu em verificar se existiam inconsistências nos dados dos jogadores. Para isso, fizemos uma query que identificava se para cada PlayerName existia mais que um campo diferente de Born, Height, Hand e LinkPlayer. Com isto notámos que existiam 7 pares de jogadores, com o mesmo nome, sendo o LinkPlayer (o URL que faz corresponder com o site de onde os dados são provenientes) o identificador único.

<pre> PlayerName : "Enrique Pena" Inconsistencias : Object Born : Array (1) 0: "" Height : Array (1) 0: "NA" Hand : Array (2) 0: "Right-Handed, Unknown Backhand" 1: "null" LinkPlayer : Array (2) 0: "https://www.atptour.com/en/players/enrique-pena/p81z/player-activity?y..." 1: "https://www.atptour.com/en/players/enrique-pena/p386/player-activity?y..." Contagens : Object Born : 1 Height : 1 Hand : 2 LinkPlayer : 2 </pre>	<pre> PlayerName : "Mark Kovacs" Inconsistencias : Object Born : Array (1) 0: "" Height : Array (1) 0: "NA" Hand : Array (1) 0: "null" LinkPlayer : Array (2) 0: "https://www.atptour.com/en/players/mark-kovacs/kb22/player-activity?ye..." 1: "https://www.atptour.com/en/players/mark-kovacs/k678/player-activity?ye..." Contagens : Object Born : 1 Height : 1 Hand : 1 LinkPlayer : 2 </pre>
--	---

Figura 2 - Jogadores com diferenças de dados;

Com base nisto, criámos uma coleção, denominada de players_collection, onde cada jogador do PlayerName foi inserido, com base no LinkPlayer único. Isto resultou numa coleção com 9.960 registos (jogadores).

```
_id: ObjectId('690dbbfbe0e82d0c3b50e01e')
PlayerName : "Thiago Alves"
Born : "Sao Jose do Rio Preto, Brazil"
Height : 178
Hand : "Right-Handed, Two-Handed Backhand"
LinkPlayer : "https://www.atptour.com/en/players/thiago-alves/a410/player-activity?y..."
```

```
_id: ObjectId('690dbbfbe0e82d0c3b50e01f')
PlayerName : "Igor Rud"
Born : ""
Height : "NA"
Hand : "Right-Handed, Unknown Backhand"
LinkPlayer : "https://www.atptour.com/en/players/igor-rud/r984/player-activity?year=..."
```

```
_id: ObjectId('690dbbfbe0e82d0c3b50e020')
PlayerName : "Mario Zavala"
Born : ""
Height : "NA"
```

Figura 3 - Exemplo de registos da coleção criada *players_collection*;

Após a verificação anterior, olhamos para o campo Oponent. Este campo representa também jogadores, no entanto apresentava nomes que não se encontravam presentes no PlayerName. Assim, criámos uma nova coleção com o nome *oponents_collection*, em que foram registados todos os nomes únicos deste campo.

Dado que para estes jogadores não existe informação relativamente aos campos Born, Hand, Height ou LinkPlayer, esta tabela consiste apenas no campo PlayerName. Isto resultou na coleção *oponents_collection* com 22.659 registos. O objetivo desta coleção, numa fase posterior (já em SQL), seria agregar à tabela final de jogadores, de forma a termos uma tabela única referente a todos os diferentes jogadores (que tenham informação complementar ou não).

```
_id: ObjectId('690e1f08e0e82d0c3b515f89')
PlayerName : "Marcelo Zormann"
```

```
_id: ObjectId('690e1f08e0e82d0c3b515f8a')
PlayerName : "Kartik Sharma"
```

```
_id: ObjectId('690e1f08e0e82d0c3b515f8b')
PlayerName : "Muhammad Abid"
```

Figura 4 - Exemplo de registos da coleção criada *oponents_collection*;

Além disto, tivemos que perceber o que distinguia um torneio (mais especificamente edição de torneio, sendo que cada evento pode ter várias edições). Com a utilização de algumas queries, notámos que existiam casos de torneios, na mesma data, com o mesmo nome, que representavam torneios diferentes.

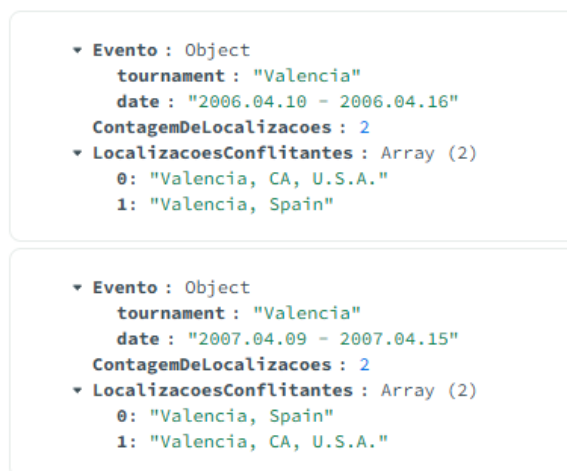


Figura 5 - Caso dos torneios com o nome 'Valencia' na mesma data em locais distintos;

Validámos se estes casos eram realmente torneios distintos, porque tanto os registos online (Torneio Challenger (nível inferior) em Valencia, EUA, 10-16 Abril 2006, e circuito ATP em Valencia, Espanha) como da base de dados comprovam isso, visto existirem duas finais distintas.

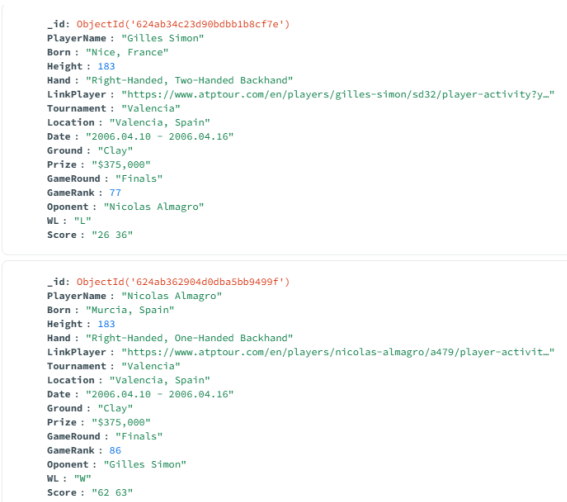


Figura 6 – 2 finais distintas para os torneios 'Valencia' em 2006;

Reparámos ainda que tínhamos torneios em que o Prize (prémio do torneio) estava inconsistente e o caso de um único torneio com Ground inconsistente (jogos do mesmo nome de torneio e data com chão diferente).

▼ Evento : Object tournament : "Lyon" location : "Lyon, France" date : "2016.06.06 - 2016.06.12" ▼ PremiosInconsistentes : Array (2) 0: "\$64,000" 1: "€64,000"
▼ Evento : Object tournament : "Murcia" location : "Murcia, Spain" date : "2019.04.08 - 2019.04.14" ▼ PremiosInconsistentes : Array (2) 0: "€46,600" 1: "\$46,600"
▼ Evento : Object tournament : "Biella" location : "Biella, Italy" date : "2018.09.17 - 2018.09.23" ▼ PremiosInconsistentes : Array (2) 0: "\$43,000" 1: "€43,000"

Figura 7 - Torneios com registos de prémios inconsistentes;

▼ Evento : Object tournament : "AUT V RSA 1RD" location : "South Africa" date : "1996.02.05 - 1996.02.11" ▼ PisosInconsistentes : Array (2) 0: "Grass" 1: "Hard"
--

Figura 8 - Apenas 1 torneio com registos de chão inconsistente;

Por estas razões, ao criarmos uma coleção de torneios, optámos por registar os torneios, tendo por base 3 colunas identificadoras: Tournament, Date e Location, utilizando o primeiro valor correspondente do Ground e Prize para os identificar. Com esta escolha poderá ter acontecido que o torneio específico ("AUT V RSA 1RD"), tenham ficado associados ao tipo de ground errado. Isto resultou na coleção tournaments_collection com 22.235 registos (eventos de torneio únicos).

_id: ObjectId('690e2553e0e82d0c3b51b80c') TournamentName : "Lyon" Date : "1992.10.19 - 1992.10.25" Location : "Lyon, France" Ground : "Carpet" Prize : "\$550,000"
_id: ObjectId('690e2553e0e82d0c3b51b80d') TournamentName : "Italy F20" Date : "2008.06.30 - 2008.07.06" Location : "Italy" Ground : "Clay" Prize : "\$15,000"

Figura 9 - Exemplo de registos da coleção criada tournaments_collection;

Após isto, outra das principais questões com que nos deparámos foi se as linhas dos jogos estariam duplicadas. Isto quando ocorre o mesmo jogo (Tournament, Date, Location e GameRound) registado duas vezes, com o PlayerName: Jogador A e Oponent: Jogador B e o oposto PlayerName: Jogador B e Oponent: Jogador A, com registos de vencedor e pontuação opostos.

Primeiro, fizemos uma query para obtermos os diferentes tipos de “WL”, e reparámos que existiam registos, tanto de vitórias, como derrotas (bem como de valores vazios).

ALL RESULTS

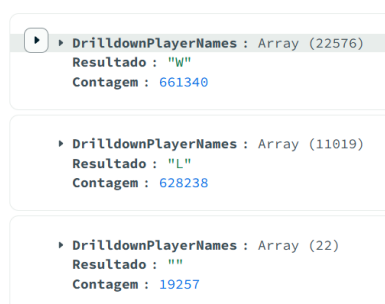


Figura 10 - Diferentes registos no campo "WL" da coleção atpplayers;

Ao fazermos *drilldown* dos valores vazios, notámos também que existem casos de jogos em que não foi registado o vencedor na maioria dos casos apresentavam o Oponent como “bye”, o que ao pesquisar sobre estes registos online, percebemos que ocorrem com a passagem automática do jogador à próxima fase. Existem ainda outros casos com o valor de “WL” vazio, mas não conseguimos identificar a razão, sendo potencialmente registos incompletos.



Figura 11 - Grande maioria dos registos com "WL" vazio representam o "Oponent": "bye";

Outra das queries que a duplicação se fez presente relativamente aos dados consistiu em pesquisar pela existência de jogos entre 2 jogadores (no caso do teste Novak Djokovic e Rafael

Nadal) na mesma data (Date) e na mesma ronda (GameRound), o que resultou na existência de 2 resultados em cada head-to-head, um para o perdedor e outro para o vencedor.

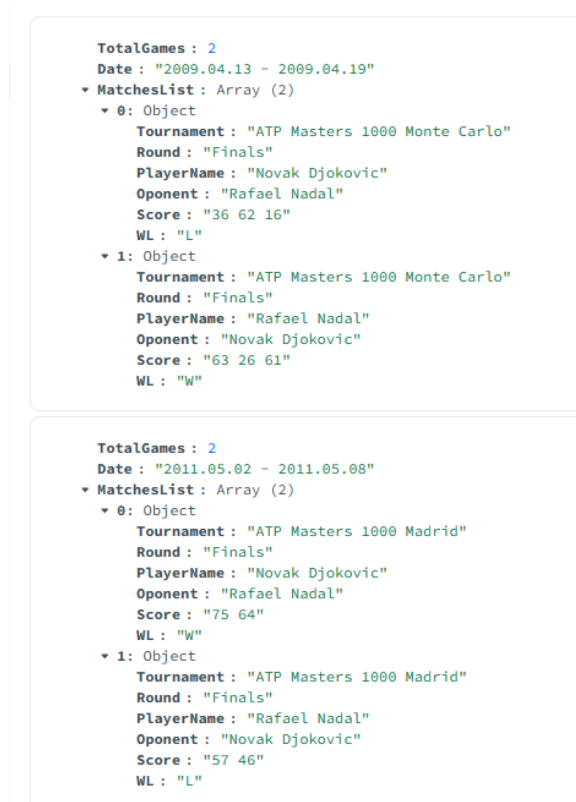


Figura 12 - Duplicação de registos de jogos na coleção atpplayers;

Além destes casos, existiam na coleção original registos duplicados do mesmo jogo, com os mesmo PlayerName e Oponent.



Figura 13 - Exemplos de registos iguais na coleção atpplayers (mesmo PlayerName e Oponent no mesmo torneio, data e localização);

Com base nisto, criámos uma coleção `unique_matches_collection`, com o objetivo de registar apenas uma linha por jogo, validando que não existe mais que um registo de jogo na mesma ronda, do mesmo torneio entre dois jogadores. Desta forma, se a mesma linha ocorrer com `PlayerName: Jogador A` e `Oponent: Jogador B` e uma outra linha com o oposto, apenas uma linha é registada (registando prioritariamente as vitórias – “WL”: “W”). Esta coleção ficou com 701.216 registos, versos os iniciais de 1.308.835, uma redução para 53% dos registos iniciais, mas garantindo assim a unicidade de jogos nos registos.

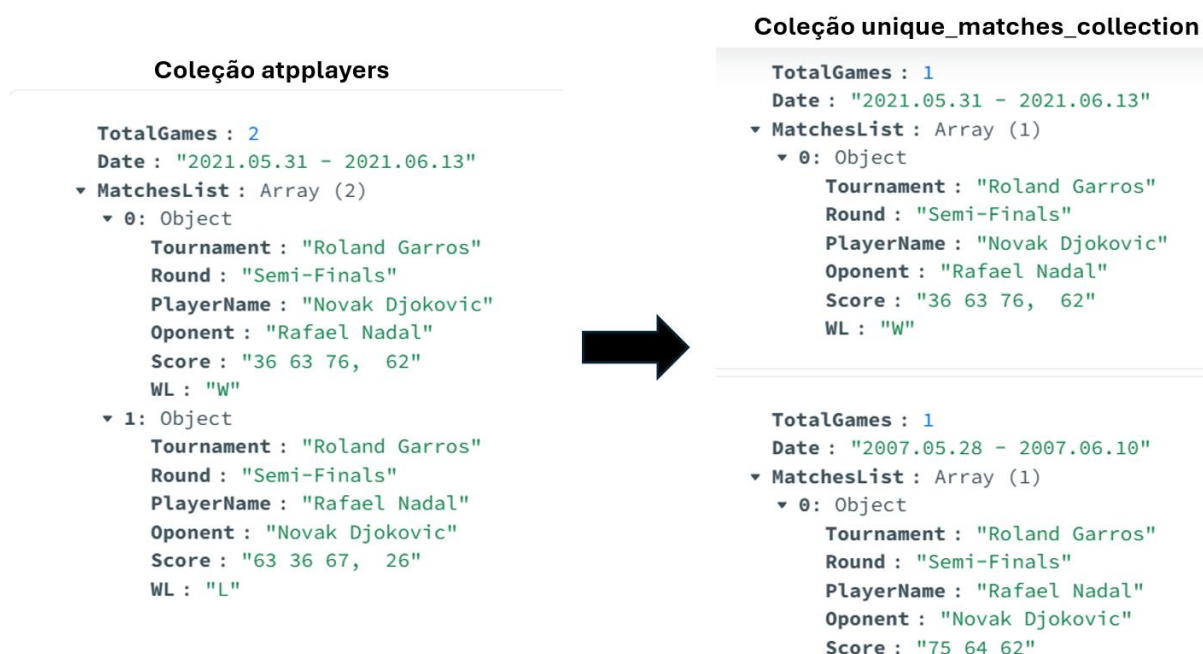


Figura 14 - Na coleção `unique_matches_collection` passamos a ter apenas um head-to-head entre 2 jogadores em cada ronda na mesma data (sem duplicação);

Exportação dos dados para csv

Após a criação das novas coleções, realizamos os comandos abaixo para exportar as 4 novas coleções para csv (`unique_matches_collection`, `players_collection`, `oponents_collection` e `tournaments_collection`):

- `mongoexport --username root --password root --authenticationDatabase admin --db atp --collection unique_matches_collection --type=csv --fields PlayerName,Born,Height,Hand,LinkPlayer,Tournament,Location,Date,Ground,Prize,GameRound,GameRank,Oponent,WL,Score --out tab_unique_matches_collection.csv`
- `mongoexport --username root --password root --authenticationDatabase admin --db atp --collection players_collection --type=csv --fields PlayerName,Born,Height,Hand,LinkPlayer --out tab_players_collection.csv`

- `mongoexport --username root --password root --authenticationDatabase admin --db atp --collection oponents_collection --type=csv --fields PlayerName --out tab_oponents_collection.csv`
- `mongoexport --username root --password root --authenticationDatabase admin --db atp --collection tournaments_collection --type=csv --fields TournamentName,Date,Location,Ground,Prize --out tab_tournaments_collection.csv`

3. Modelo relacional

Idealização do modelo relacional

Numa primeira fase, começámos por definir o que seria o modelo relacional ideal a construir para responder às queries-objetivo do trabalho.

Considerámos que o que faria mais sentido consistia em termos 4 tabelas: de jogadores, torneios, partidas/jogos, e de países (visto um dos objetivos principais seria responder às agregações de dados de jogos, jogadores e torneios por país).

Este modelo relacional consiste na criação de chaves estrangeiras que ligassem:

- ID de cada jogador (tabela `players`) a um `PlayerID` na partida (tabela `unique_matches`);
- ID de cada jogador (tabela `players`) a um `OponentID` na partida (tabela `unique_matches`);
- ID de cada torneio (tabela `tournaments`) a um `TournamentID` na partida (tabela `unique_matches`);
- ID de cada país (`country_code` da tabela `countries`) a um `country_code` a cada jogador e torneio (`country_code` das tabelas `players` e `tournaments`);

Para permitir a ligação com países foram utilizadas diferentes ferramentas, tendo como principal fonte uma tabela normalizada de países disponibilizada no enunciado do trabalho, à qual é explorada em maior detalhe no restante trabalho.

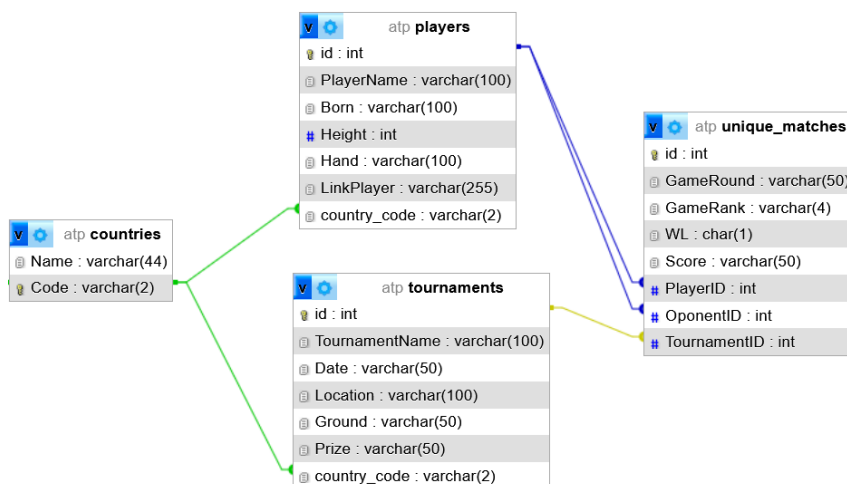


Figura 15 - Modelo relacional idealizado;

Importação de dados e criação das tabelas iniciais

Para permitir a importação dos 4 csv para SQL, foram criadas as 4 tabelas, cada uma para acomodar um dos ficheiros:

Tabela	Ação	Registos	Tipo	Agrupamento (Collation)	Tamanho	Suspensão
☐ oponents	Procurar Estrutura Pesquisar Inserir Esvaziar Eliminar	22,659	InnoDB	utf8mb4_0900_ai_ci	1.5 MB	-
☐ players	Procurar Estrutura Pesquisar Inserir Esvaziar Eliminar	9,960	InnoDB	utf8mb4_0900_ai_ci	16.0 KB	-
☐ tournaments	Procurar Estrutura Pesquisar Inserir Esvaziar Eliminar	22,235	InnoDB	utf8mb4_0900_ai_ci	2.5 MB	-
☐ unique_matches	Procurar Estrutura Pesquisar Inserir Esvaziar Eliminar	701,216	InnoDB	utf8mb4_0900_ai_ci	199.7 MB	-
4 tabela(s)	Soma	756,070	InnoDB	utf8mb4_0900_ai_ci	203.7 MB	0 Bytes

Figura 16 - Tabelas criadas para a importação dos ficheiros csv;

A tabela oponents consistia, tal como a coleção oponents_collection apenas num campo de PlayerName, ao qual foi acrescentado um ID AUTO_INCREMENT.

#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra	Ação
☐ 1	id	int			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
☐ 2	PlayerName	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais

Figura 17 - Tabela inicial oponents;

A tabela players apresenta as mesmas informações da coleção players_collection, ao qual foi acrescentado um ID AUTO_INCREMENT. É de notar que o campo Height teve de ser criado como varchar, visto os dados apresentarem valor em texto “null”.

Estrutura da tabela		Visão de relação(ões)							
#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra	Ação
☐ 1	id	int			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
☐ 2	PlayerName	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 3	Born	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 4	Height	varchar(4)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 5	Hand	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 6	LinkPlayer	varchar(255)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais

Figura 18 - Tabela inicial players;

A tabela tournaments apresenta as mesmas informações da tournaments_collection, ao qual foi acrescentado um ID AUTO_INCREMENT. Os campos Location e Prize tiveram de ser criados como varchar, visto os dados apresentarem valores em texto.

#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra	Ação
☐ 1	id	int			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
☐ 2	TournamentName	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 3	Date	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 4	Location	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 5	Ground	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 6	Prize	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais

Figura 19 - Tabela inicial tournaments;

Finalmente, a tabela unique_matches apresenta as mesmas informações da unique_matches_collection, com o ID AUTO_INCREMENT. Mais uma vez os restantes campos foram criados como varchar para permitir os registos no formato original.

#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra	Ação
☐ 1	id	int			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
☐ 2	PlayerName	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 3	Born	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 4	Height	varchar(4)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 5	Hand	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 6	LinkPlayer	varchar(255)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 7	Tournament	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 8	Location	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 9	Date	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 10	Ground	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 11	Prize	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 12	GameRound	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 13	GameRank	varchar(4)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 14	Oponent	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 15	WL	char(1)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐ 16	Score	varchar(50)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais

Figura 20 - Tabela inicial unique_matches;

Limpeza de dados e criação de relações

Construção da tabela final de jogadores (tabela players)

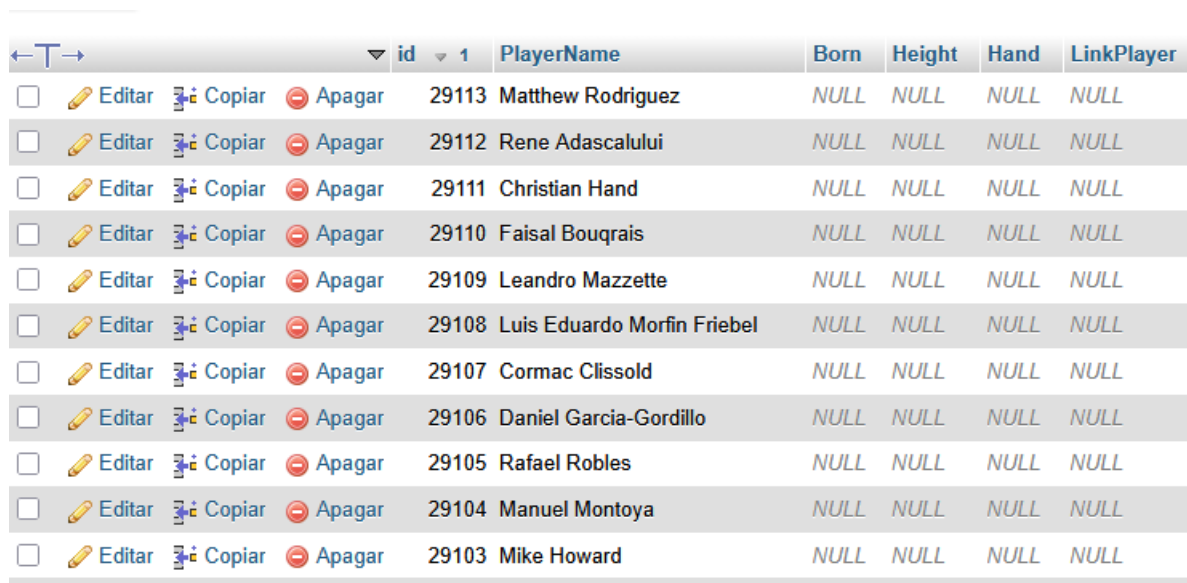
Após inserir todas as tabelas exportadas do mongoDB, o primeiro passo feito passou por criar uma tabela final de jogadores, que agregasse tanto os jogadores presentes no campo PlayerName como os jogadores no campo Oponent.

De forma a garantir isto, foi realizado um comando de INSERT para atualizar a tabela players, que inserisse apenas os nomes que não estavam já presentes na tabela players.

Notámos ainda que esta abordagem, e a própria estrutura dos dados originais tinham uma limitação. Poderiam ocorrer casos de jogadores da tabela Oponents que partilhassem o mesmo nome, tal como notámos nas ocorrências de PlayerName. No entanto, visto não termos mais detalhe sobre os jogadores que aparecem no campo Oponent (um Link ou qualquer outra característica identificadora), optámos por proceder desta forma.

- INSERT INTO players (PlayerName)
SELECT PlayerName from oponents
WHERE PlayerName NOT IN (SELECT DISTINCT PlayerName FROM players);

Este comando acrescentou 12.730 jogadores à tabela players, dos quais não temos qualquer informação tirando o nome.



	id	PlayerName	Born	Height	Hand	LinkPlayer
☐ Editar Copiar Apagar	29113	Matthew Rodriguez	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29112	Rene Adascalului	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29111	Christian Hand	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29110	Faisal Bouqrais	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29109	Leandro Mazzette	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29108	Luis Eduardo Morfin Friebe	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29107	Cormac Clissold	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29106	Daniel Garcia-Gordillo	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29105	Rafael Robles	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29104	Manuel Montoya	NULL	NULL	NULL	NULL
☐ Editar Copiar Apagar	29103	Mike Howard	NULL	NULL	NULL	NULL

Figura 21 - Exemplos de registos de jogadores inseridos na tabela players provenientes da tabela oponents;

Correspondência de torneios e jogadores com países

Como um dos objetivos principais do trabalho consistia em agrupar dados por país, o próximo passo consistiu em criar a tabela de países (countries), com base na listagem normalizada de países disponibilizada no enunciado do trabalho. Foram também criadas as colunas `country_code` nas tabelas `players` e `tournaments` e a correspondente chave estrangeira com `country_code` na tabela `countries` para garantir a ligação.

#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra	Ação
☐	1 Name	varchar(44)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐	2 Code	varchar(2)	utf8mb4_0900_ai_ci		Não	Nenhum			Alterar Eliminar Mais

Figura 22 - Estrutura da nova tabela criada (countries);

Depois de criada a tabela, como tínhamos como objetivo fazer a correspondência entre torneios, o país de realização, os jogadores e os países de nascimento, decidimos criar uma tabela temporária que agregasse todas as ocorrências distintas de Born (da tabela `players`) e Location (da tabela `tournaments`). Criámos ainda a tabela `country_location_match` para este propósito, de forma a, após limpeza dos dados, conseguirmos fazer a correspondência com o país certo. Criamos ainda a chave estrangeira na coluna `country_code` para garantir depois uma ligação à tabela `countries`.

#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Comentários	Extra	Ação
☐	1 location	varchar(100)	utf8mb4_0900_ai_ci		Não	Nenhum			Alterar Eliminar Mais
☐	2 extracted_country	varchar(100)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais
☐	3 country_code	varchar(2)	utf8mb4_0900_ai_ci		Sim	NULL			Alterar Eliminar Mais

Figura 23 - Estrutura da nova tabela criada `country_location_match`;

Para construir essa tabela incluímos uma coluna com a informação em bruto, também incluímos uma segunda que incluisse já uma tentativa inicial de limpeza (selecionando o valor à direita da vírgula – pois a maioria dos campos encontrava-se preenchido seguindo “Lisbon, Portugal”), e uma terceira coluna com o código do país da tabela `countries`, servindo como chave estrangeira.

Isto resultou numa tabela com 4.192 diferentes localizações a tratar.

1	>	>>	Número de registos: 25	Filtrar registos: Pesquisar esta tabela	Or
Opções extra					
		location	extracted_country	country_code	
☐	Editar	Copiar	Apagar	's-Hertogenbosch, Netherlands	Netherlands
☐	Editar	Copiar	Apagar	Aachen, Germany	Germany
☐	Editar	Copiar	Apagar	Aalen, Germany	Germany
☐	Editar	Copiar	Apagar	Aalst, Belgium	Belgium
☐	Editar	Copiar	Apagar	Aarhus, Denmark	Denmark
☐	Editar	Copiar	Apagar	Abidjan, CIV	CIV
☐	Editar	Copiar	Apagar	Abidjan, Ivory Coast	Ivory Coast
☐	Editar	Copiar	Apagar	Abuja, Nigeria	Nigeria
☐	Editar	Copiar	Apagar	Abymes, Guadeloupe	Guadeloupe
☐	Editar	Copiar	Apagar	Acapulco, Mexico	Mexico
☐	Editar	Copiar	Apagar	Accra, Ghana	Ghana
☐	Editar	Copiar	Apagar	Acquapendente, Italy	Italy
☐	Editar	Copiar	Apagar	Acqui Terme, Italy	Italy

Figura 24 - Exemplo de registos iniciais da tabela `country_location_match`;

Numa fase inicial, começamos por fazer UPDATE na tabela de todas as localizações em que o `extracted_country` cruzava diretamente com o país da tabela `countries`, com o objetivo de atualizar a chave estrangeira. Isto resultou diretamente na alocação correta de uma grande fatia dos registos

De forma a priorizar as ações a tomar nas restantes correspondências, realizamos uma nova query que indicasse o número de jogadores e torneios associados a cada `extracted_country`.

✓ A mostrar registos de 0 - 24 (198 total. A consulta demorou 0.1733 s)

```
SELECT cl.extracted_country, COUNT(DISTINCT t.id) AS total_players AS p ON p.born = cl.location WHERE cl.country_code
```

Perfil | Editar em linha | [Editar] | [Explicar SQL] | [Criar código PHP]

1 > >> Mostrar tudo Número de registos: 25

Opções extra

extracted_country	total_torneios	total_jogadores
U.S.A.	2241	25
Great Britain	355	20
Russia	175	88
England	131	84
Bosnia & Herzegovina	74	0
South Korea	65	8
Korea	64	14
Chinese Taipei	45	5
Bolivia	45	5
Vietnam	42	2
Venezuela	41	9
U.A.E.	40	0
Slovak Republic	37	4
Iran	34	1
U.S.A.	26	1
USA	24	248
Macedonia	18	3
Yugoslavia	14	5
Reunion Island	11	0
Salvador	11	0

Figura 25 - Query que serviu de base para a priorização das queries UPDATE de limpeza;

Após esta consulta, realizamos algumas queries de limpeza da coluna `extracted_country`, para, após isso, podermos fazer um novo comando de UPDATE da tabela.

Como o processo de correção de localizações, estava a ser demorado e tinha algumas falhas, optámos por uma última tentativa de limpeza das localizações associados aos jogos e torneios, tendo como base o ficheiro csv presente no site <https://simplemaps.com/data/world-cities>. Para esta fase da limpeza criamos uma nova tabela `worldcities_small`, na qual foram importadas apenas as colunas relevantes do csv e foram retiradas as linhas associadas a

idades não correspondentes com o código de países da tabela countries (para podermos criar uma chave estrangeira entre o iso2 – código de país e country_code).

A mostrar registos de 0 - 24 (47990 total, A consulta demorou 0.0014 segundos.)

SELECT * FROM `worldcities\_small`

Perfil

Editar em linha

Editar

Explicar SQL

Criar código PHP

Atualizar

1

>

>>

Número de registos: 25

Filtrar registos:

Pesquisar esta tabela

Ord

Opções extra

↵

→

<

Figura 26 - Exemplo de registos da tabela worldcities_small;

Após a importação desta nova tabela foram realizadas mais algumas limpezas, no entanto, este mesmo ficheiro não resolvia todos os casos. Como é possível ver na imagem abaixo, ao tentarmos cruzar alguma da informação da tabela country_location_match com as cidades de worldcities_small, surgiam certos erros (nos casos de cidades existentes em vários países), como, por exemplo, Birmingham, Amsterdam, Belgrade a serem associados aos países incorretos. De forma a evitar a correspondência com o país errado via cidade, decidimos não avançar com esta correspondência.

⚠ A seguinte frase não contém uma coluna existente. Os resultados do campo de grande, tabela de jogadores, tabelas, jogadores e jogadores não estão suportados.

✓ A mostrar registos de 0 - 24 (93 total, A consulta demorou 0.0395 segundos.) [location: ABIDJAN, CIV... - DAMASCUS, SYRIA...]

```
SELECT clm.location, clm.city, wc.country FROM country_location_match clm JOIN worldcities_small wc ON wc.city = clm.city WHERE country_code IS NULL ORDER BY `clm`.`location` ASC
```

☐ Perfil [\[Editar em linha \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Criar código PHP \]](#) [\[Actualizar \]](#)

1 > >> ☐ Mostrar tudo | Número de registos: 25 Filtrar registos: Pesquisar esta tabela | Ordenar pela chave: Nenhum

Opções extra

location	city	country
Abidjan, CIV	Abidjan	Côte d'Ivoire
Amsterdam, The Nethe	Amsterdam	Netherlands
Amsterdam, The Nethe	Amsterdam	United States
Battambang, Camboda	Battambang	Cambodia
Belgrade, SCG	Belgrade	Serbia
Belgrade, SCG	Belgrade	United States
Belgrade, Yug	Belgrade	United States
Belgrade, Yug	Belgrade	Serbia
Birmingham, Britain	Birmingham	United States
Birmingham, Britain	Birmingham	United Kingdom
Birmingham, Britain	Birmingham	United States
Birmingham, GBR	Birmingham	United States
Birmingham, GBR	Birmingham	United Kingdom
Birmingham, GBR	Birmingham	United States
Birmingham, UK	Birmingham	United States
Birmingham, UK	Birmingham	United Kingdom
Birmingham, UK	Birmingham	United States
Cairo, Egypt	Cairo	Egypt
Cairo, Egypt	Cairo	United States
Cairo, Egypt	Cairo	Egypt
Cairo, Egypt	Cairo	United States
Calgary, Alberta	Calgary	Canada
Carson, CA, United S	Carson	United States
Coatzacoalcas, Meixco	Coatzacoalcas	Mexico
Damascus, Syria	Damascus	United States

Figura 27 - Exemplo de potenciais erros que poderiam surgir no cruzamento cidade -> país (country representa o país identificado na tentativa de match);

Depois da limpeza das localizações, adicionámos o campo de country_code como chave estrangeira às tabelas players e tournaments, de forma a garantir ligação entre os países, jogadores e torneios. Seguiu-se a isso fazer a correspondência através de um comando UPDATE.

Com este processo de limpeza ficaram por fazer corresponder a um país 199 jogadores que tinham informação no campo Born de um total de 3684, ou seja 5%, e 204 torneios de 22031 com correspondência do país, menos de 1%.

✓ A mostrar registos de 0 - 24 (157 total, A consulta demorou 0.0023 segundos.)

```
SELECT * FROM players WHERE country_code IS NULL AND BORN IS NOT NULL;
```

Perfil [Editar em linha] [Editar] [Explicar SQL] [Criar código PHP] [Actualizar]

1 > >> | ☐ Mostrar tudo | Número de registos: 25 | Filtrar registos: Pesquisar esta tabela | Ordenar pela chave: Nenhum

Opções extra

	id	PlayerName	Born	Height	Hand	LinkPlayer	country_code
☐ Editar Copiar Apagar	129	Xiao Gong	Hunan	178	Left-Handed, Two-Handed Backhand	https://www.atptour.com/en/players/xiao-gong/ge09/...	NULL
☐ Editar Copiar Apagar	174	Luca Tomasetto	Ciri	185	Right-Handed, Two-Handed Backhand	https://www.atptour.com/en/players/luca-tomasetto/...	NULL
☐ Editar Copiar Apagar	227	Przemyslaw Stec	Priemysl	NULL	NULL	https://www.atptour.com/en/players/przemyslaw-stec...	NULL
☐ Editar Copiar Apagar	275	Jesus Francisco Felix	Santo Domingo, Dominican Rep.	178	Right-Handed, One-Handed Backhand	https://www.atptour.com/en/players/jesus-francisco...	NULL
☐ Editar Copiar Apagar	348	Niklas Gutttau	Lbeck	178	Left-Handed, Two-Handed Backhand	https://www.atptour.com/en/players/niklas-gutttau/g...	NULL
☐ Editar Copiar Apagar	352	Jayden Court	Monto	188	Right-Handed, One-Handed Backhand	https://www.atptour.com/en/players/jayden-court/c0...	NULL
☐ Editar Copiar Apagar	357	Zhao Cai	sichuan suining	175	Right-Handed, Two-Handed Backhand	https://www.atptour.com/en/players/zhao-cai/cf05/p...	NULL
☐ Editar Copiar Apagar	442	Zhaoyi Cao	HEILONGJIANG	185	Right-Handed, Unknown Backhand	https://www.atptour.com/en/players/zhaoyi-cao/cg42...	NULL

Figura 28 - Exemplos de jogadores que ficaram por fazer corresponder com país;

✓ A mostrar registos de 0 - 24 (195 total, A consulta demorou 0.0041 segundos.)

```
SELECT * FROM `tournaments` where country_code is null;
```

Perfil [Editar em linha] [Editar] [Explicar SQL] [Criar código PHP] [Actualizar]

1 > >> | ☐ Mostrar tudo | Número de registos: 25 | Filtrar registos: Pesquisar esta tabela | Ordenar pela chave: Nenhum

Opções extra

	id	TournamentName	Date	Location	Ground	Prize	country_code
☐ Editar Copiar Apagar	304	Yugoslavia F3	1998.05.25 - 1998.05.31	Yugoslavia	Clay	\$10,000	NULL
☐ Editar Copiar Apagar	453	M15 Cairo	2021.07.26 - 2021.08.01	Cairo, Egypt	Clay	\$15,000	NULL
☐ Editar Copiar Apagar	623	ESP V USR WGPO	1990.09.17 - 1990.09.23	Soviet Union	Carpet		NULL
☐ Editar Copiar Apagar	654	Syria F2	2009.07.13 - 2009.07.19	Syria	Hard	\$15,000	NULL
☐ Editar Copiar Apagar	780	ESP v. NED WG Qtrs	2004.04.05 - 2004.04.11	Palma de Mallorca	Clay		NULL
☐ Editar Copiar Apagar	955	GBR v SWE WG Rd 1	2002.02.04 - 2002.02.10	Birmingham, GBR	Carpet		NULL
☐ Editar Copiar Apagar	968	Great Britain F10	2002.10.07 - 2002.10.13	TBC	Hard	\$15,000	NULL
☐ Editar Copiar Apagar	1087	Serbia & Montenegro F4	2006.08.07 - 2006.08.13	Serbia & Montenegro	Clay	\$10,000	NULL
☐ Editar Copiar Apagar	1150	Mexico F6A	2004.05.17 - 2004.05.30	Coatzacoalcos, Meixco	Hard	\$10,000	NULL
☐ Editar Copiar Apagar	1452	PHI v KUW AOGII PO	2002.04.01 - 2002.04.07	Manilla	Carpet		NULL
☐ Editar Copiar Apagar	1674	Martinique	1990.03.05 - 1990.03.11	Martinique, Lesser Antilles	Hard	\$50,000	NULL
☐ Editar Copiar Apagar	1684	BRA v. CAN AGI 2nd Rd	2007.04.02 - 2007.04.08	Brazi	Clay		NULL

Figura 29 - Exemplos de torneios que ficaram por fazer corresponder com país;

Ligação entre jogos, torneios e jogadores

Seguiu-se à limpeza de países a criação das ligações entre as tabelas de unique_matches, tournaments e players, através da criação de 3 novas colunas (TournamentID, PlayerID, OponentID) com 3 chaves estrangeiras.

Ações	Propriedades da restrição	Coluna	Restrição de chave estrangeira (InnoDB)		
			Base de Dados	Tabela	Coluna
Eliminar	fk_matchoponent_playerid ON DELETE RESTRICT ON UPDATE RESTRICT	OponentID + Adicionar coluna	atp	players	id
Eliminar	fk_matchplayer_playerid ON DELETE RESTRICT ON UPDATE RESTRICT	PlayerID + Adicionar coluna	atp	players	id
Eliminar	fk_matchtournament_tourn ON DELETE RESTRICT ON UPDATE RESTRICT	TournamentID + Adicionar coluna	atp	tournaments	id

Figura 30 - chaves estrangeiras entre jogos, torneios e jogadores;

Depois de criadas as colunas e chaves estrangeiras, foi necessário atualizar os campos de ID de forma a fazer corresponder com o ID de jogador ou torneio correto. Para isso foram criados índices nas colunas de pesquisa que iriam ser utilizadas para os comandos de UPDATE das 3 tabelas: LinkPlayer, PlayerName, Oponent, Tournament (na tabela unique_matches), TournamentName (na tabela tournaments), Date e Location.

Após isto retirámos as colunas que já não eram necessárias da tabela unique_matches (informação relativa ao torneio e jogo) e realizamos algumas queries finais de limpeza:

- Alterado score para NULL nos casos em que estava vazio ou escrito “null”;
- Alterado WL para NULL nos casos em que estava vazio;
- Removendo da tabela players o jogador com o PlayerName “vazio”, o que aparece apenas num número limitado de jogos como oponente (7 jogos, em que o OponentID para a estar a NULL);

Limpeza final de dados

Numa tentativa de tornar os dados mais limpos, fizemos um conjunto de limpezas finais em alguns campos adicionais:

1. Alterado o campo “Height” para NULL quando “NA”;
2. Alterado o campo “Height” para INT;
3. Alterado o campo “Height” para NULL quando < 100 ou > 300;
4. Alterado o campo “Hand” para NULL quando “NA”;
5. Alterado o campo “Born” para NULL quando “” (vazio);

Modelo relacional final

Abaixo apresentamos o modelo relacional final, que, ao contrário do modelo apresentado no início, inclui também as tabelas de apoio worldcities_small e country_location_match, com a chave estrangeira para o country_code da tabela countries.

Apesar de serem apenas elementos provisórios do modelo relacional, mantivemos no modelo relacional numa ótica de poderem servir como base para uma correspondência futura mais completa entre países e jogadores/torneios.

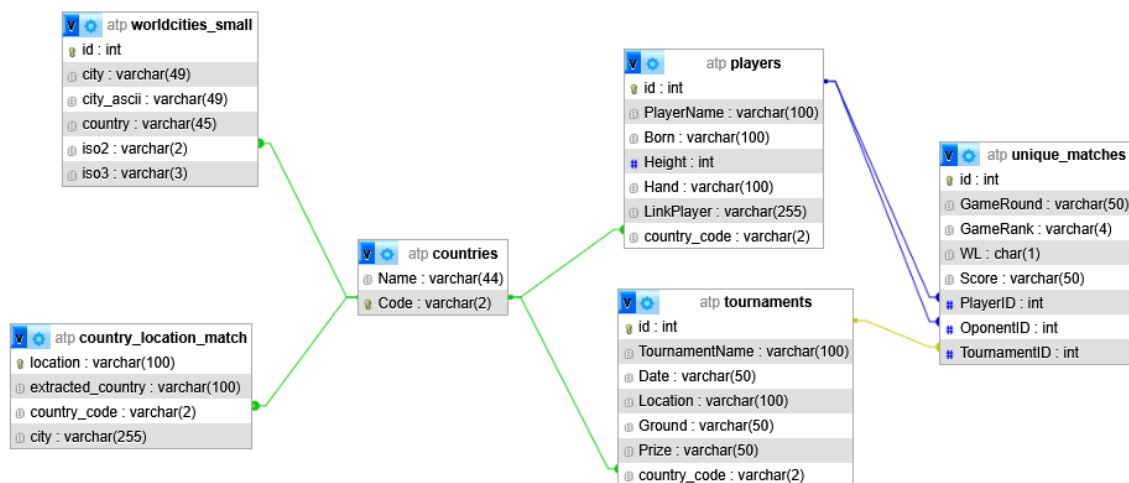


Figura 31 - Modelo relacional final;

4. Queries SQL

Query 1 - Jogadores, torneios e jogos por país

Para realizar primeira query começámos por criar uma view para cada contagem (jogadores, torneios e jogos), de forma a facilitar a construção da query final.

Para a primeira VIEW, da contagem de jogadores foi utilizado o comando:

- CREATE VIEW player_counts AS
SELECT country_code, COUNT(id) AS TotalPlayers
FROM players
GROUP BY country_code;

Para a segunda VIEW, de contagem de torneios foi utilizado o comando:

- CREATE VIEW tournament_counts AS
SELECT country_code, COUNT(id) AS TotalTournaments
FROM tournaments
GROUP BY country_code;

Para a terceira VIEW, de contagem de jogos, tivemos de definir que íamos excluir da contagem os jogos com WL vazio ou jogados contra o oponente “bye” (com ID 28049), visto serem jogos que na realidade não ocorreram, ou que não temos informação consistente para identificar se ocorreram ou quem foi o vencedor.

- CREATE VIEW game_counts AS
SELECT t.country_code, COUNT(um.id) AS TotalGames
FROM unique_matches um
JOIN tournaments t ON t.id = um.TournamentID
WHERE um.WL IS NOT NULL AND um.OponentID <> 28049
GROUP BY t.country_code;

A query final consistiu na utilização de um COALESCE, permitindo assim devolver valores a 0. Encontra-se abaixo apresentada:

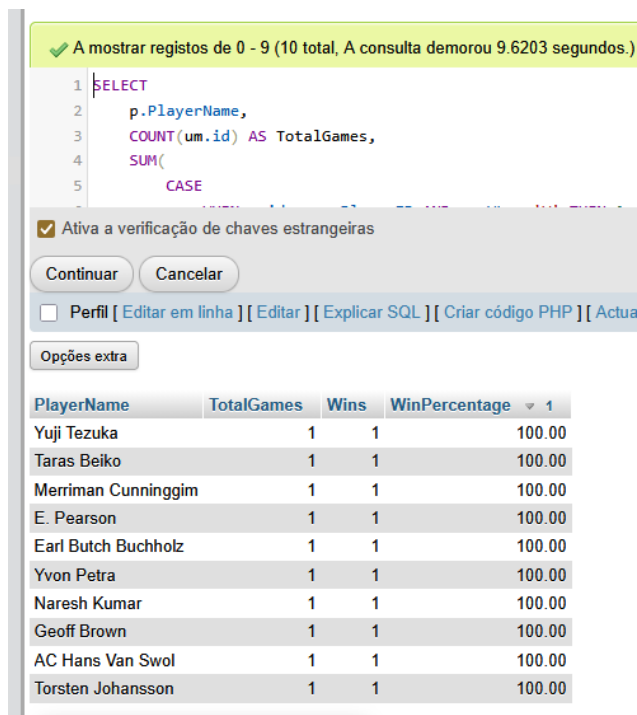
- SELECT
c.name AS CountryName,
COALESCE(p.TotalPlayers,0) AS TotalPlayers,
COALESCE(t.TotalTournaments,0) AS TotalTournaments,
COALESCE(g.TotalGames,0) AS TotalGames
FROM countries c
LEFT JOIN player_counts p ON c.code = p.country_code
LEFT JOIN tournament_counts t ON c.code = t.country_code
LEFT JOIN game_counts g ON c.code = g.country_code
ORDER BY c.name;

CountryName	TotalPlayers	TotalTournaments	TotalGames
Afghanistan	0	0	0
Åland Islands	0	0	0
Albania	0	0	0
Algeria	3	35	914
American Samoa	0	0	0
Andorra	0	13	349
Angola	0	0	0
Anguilla	0	0	0
Antarctica	0	0	0
Antigua and Barbuda	0	0	0
Argentina	169	438	12407
Armenia	2	7	216
Aruba	0	4	124
Australia	153	578	21357
Austria	60	357	10494
Azerbaijan	0	9	277
Bahamas	5	20	114
Bahrain	0	12	374
Bangladesh	0	2	62
Barbados	2	16	110
Belarus	20	68	1339
Belgium	41	229	5986
Belize	0	0	0
Benin	0	0	0
Bermuda	0	17	483

Figura 32 - Resultado da 1ª query;

Query 2 – Top 10 jogadores por winning rate

Para responder a esta query começamos por fazer a query diretamente e tínhamos um top 10 de jogadores com apenas um jogo e winrate de 100%.



✓ A mostrar registos de 0 - 9 (10 total, A consulta demorou 9.6203 segundos.)

```
1 SELECT
2   p.PlayerName,
3   COUNT(um.id) AS TotalGames,
4   SUM(
5     CASE
```

☒ Ativa a verificação de chaves estrangeiras

Continuar Cancelar

☐ Perfil [Editar em linha] [Editar] [Explicar SQL] [Criar código PHP] [Actualizar]

Opções extra

PlayerName	TotalGames	Wins	WinPercentage
Yuji Tezuka	1	1	100.00
Taras Beiko	1	1	100.00
Merriman Cunninggim	1	1	100.00
E. Pearson	1	1	100.00
Earl Butch Buchholz	1	1	100.00
Yvon Petra	1	1	100.00
Naresh Kumar	1	1	100.00
Geoff Brown	1	1	100.00
AC Hans Van Swol	1	1	100.00
Torsten Johansson	1	1	100.00

Figura 33 - Primeira tentativa de resolução da query 2;

Desta forma tivemos de contar qual seria o valor mínimo de jogos jogados para serem considerados.

Para chegar a isso realizámos uma query que indicasse o número de jogos por jogador, com a qual notámos um grande volume de jogos em certos jogadores.

PlayerID	PlayerName	TotalGames ▾ 1
7480	Jimmy Connors	1584
4748	Roger Federer	1568
4039	Rafael Nadal	1360
3640	Ivan Lendl	1322
1448	Guillermo Vilas	1289
2613	Novak Djokovic	1277
19956	David Ferrer	1217
59	Paolo Lorenzi	1199
5762	Teymuraz Gabashvili	1182
1150	Andre Agassi	1167
7107	Feliciano Lopez	1165
1619	Tommy Robredo	1150
4314	Ruben Ramirez Hidalgo	1146
3610	Fernando Verdasco	1134
2478	Andrea Arnaboldi	1128
2706	Sergiy Stakhovsky	1118
7070	Fabrice Santoro	1115
3254	Gilles Simon	1113
4590	Andreas Seppi	1111
9267	Go Soeda	1106
739	Ti Chen	1106
8396	Mikhail Youzhny	1106
8469	John McEnroe	1093
895	Lukas Rosol	1086
23204	Tomas Berdych	1084

Figura 34 - Número de jogos por jogador, ordenando do maior para menor;

No entanto, para garantir a inclusão de jogadores com menos experiência, realizamos após isso uma outra query que calculasse a média de jogos jogados por jogador (excluindo jogos sem resultados – “WL” NULL ou contra o oponente “bye”), que resultou em 60.13 jogos.

mostrar Caixa do query

⚠ A seleção atual não contém uma coluna exclusiva. Os recursos de edição de grade, caixa de seleção, Editar,

✓ A mostrar registos de 0 - 0 (1 total, A consulta demorou 9.2817 segundos.)

```
SELECT ROUND(COUNT(um.id) / COUNT(DISTINCT p.id), 2) AS AvgGamesPerPlayer FROM players p JOIN unique_matches um ON p.id = um.PlayerID OR p.id = um.OponentID
```

☐ Perfil [[Editar em linha](#)] [[Editar](#)] [[Explicar SQL](#)] [[Criar código PHP](#)] [[Atualizar](#)]

☐ Mostrar tudo | Número de registos: 25 ▾ Filtrar registos:

Opções extra

AvgGamesPerPlayer

60.13

☐ Mostrar tudo | Número de registos: 25 ▾ Filtrar registos:

Figura 35 - Média de jogos por jogador;

Aplicamos assim um limite mínimo de 60 jogos à query inicial:

- SELECT
p.PlayerName,
COUNT(um.id) AS TotalGames,
SUM(CASE
WHEN p.id = um.PlayerID AND um.WL = 'W' THEN 1
WHEN p.id = um.OponentID AND um.WL = 'L' THEN 1
ELSE 0
END) AS Wins,
ROUND(100 * SUM(CASE
WHEN p.id = um.PlayerID AND um.WL = 'W' THEN 1
WHEN p.id = um.OponentID AND um.WL = 'L' THEN 1
ELSE 0
END) / COUNT(um.id), 2) AS WinPercentage
FROM players p
JOIN unique_matches um ON p.id = um.PlayerID OR p.id = um.OponentID
GROUP BY p.PlayerName
HAVING TotalGames > 60
ORDER BY WinPercentage DESC
LIMIT 10;

PlayerName	TotalGames	Wins	WinPercentage ▼ 1
Bjorn Borg	824	659	79.98
John McEnroe	1145	891	77.82
Ivan Lendl	1399	1079	77.13
Jimmy Connors	1665	1281	76.94
Roger Federer	1688	1281	75.89
Novak Djokovic	1396	1059	75.86
Rafael Nadal	1501	1130	75.28
Carlos Alcaraz	161	119	73.91
Kent Carlsson	183	135	73.77
Paul Jubb	144	105	72.92

Figura 36 - Resultado 2ª query;

Query 3 – Top 10 jogadores esquerdinos por winning rate Grand Slam

Para a terceira query, foi feita uma query com um cálculo semelhante, em que foram filtrados os jogadores esquerdinos (“Left-Handed, Unknown Backhand”, “Left-Handed, One-Handed Backhand”, “Left-Handed, Two-Handed Backhand”) e jogos de torneios Grand Slam (“Roland Garros”, “Australian Open”, “US Open”, “Wimbledon”).

- ```
SELECT
 p.PlayerName,
 COUNT(um.id) AS TotalGames,
 SUM(CASE
 WHEN p.id = um.PlayerID AND um.WL = 'W' THEN 1
 WHEN p.id = um.OponentID AND um.WL = 'L' THEN 1
 ELSE 0
 END) AS Wins,
 ROUND(100 * SUM(CASE
 WHEN p.id = um.PlayerID AND um.WL = 'W' THEN 1
 WHEN p.id = um.OponentID AND um.WL = 'L' THEN 1
 ELSE 0
 END) / COUNT(um.id), 2) AS WinPercentage
FROM players p
```

```

JOIN unique_matches um
 ON p.id = um.PlayerID OR p.id = um.OponentID
JOIN tournaments t
 ON t.id = um.TournamentID
WHERE p.Hand IN ('Left-Handed, Unknown Backhand',
 'Left-Handed, One-Handed Backhand',
 'Left-Handed, Two-Handed Backhand')
AND t.TournamentName IN ('Roland Garros',
 'Australian Open',
 'US Open',
 'Wimbledon')
AND um.WL IS NOT NULL
AND um.OponentID <> 28049
GROUP BY p.ID, p.PlayerName
HAVING TotalGames > 60
ORDER BY WinPercentage DESC
LIMIT 10;

```

| PlayerName        | TotalGames | Wins | WinPercentage ▾ 1 |
|-------------------|------------|------|-------------------|
| Rafael Nadal      | 341        | 299  | 87.68             |
| Jimmy Connors     | 283        | 233  | 82.33             |
| John McEnroe      | 208        | 169  | 81.25             |
| Guillermo Vilas   | 178        | 134  | 75.28             |
| Goran Ivanisevic  | 160        | 110  | 68.75             |
| Thomas Muster     | 116        | 78   | 67.24             |
| Andres Gomez      | 93         | 62   | 66.67             |
| Marcelo Rios      | 77         | 51   | 66.23             |
| Henri Leconte     | 125        | 81   | 64.80             |
| Fernando Verdasco | 184        | 114  | 61.96             |

Figura 37 - Resultado 3ª query;

## Query 4 – Top 5 jogadores por winning rate em chão “Hard”

A última query mantém-se semelhante às anteriores, no entanto faz uma contagem apenas das vitórias, filtrando pelos jogos de torneios com o chão “Hard”.

- ```
SELECT
  p.PlayerName,
  SUM( CASE
    WHEN p.id = um.PlayerID AND um.WL = 'W' THEN 1
    WHEN p.id = um.OponentID AND um.WL = 'L' THEN 1
    ELSE 0
  END) AS Wins
FROM players p
JOIN unique_matches um
  ON p.id = um.PlayerID OR p.id = um.OponentID
JOIN tournaments t
  ON t.id = um.TournamentID
WHERE t.Ground = "Hard" AND um.WL IS NOT NULL
  AND um.OponentID <> 28049
GROUP BY p.ID, p.PlayerName
ORDER BY Wins DESC
LIMIT 5;
```

PlayerName	Wins
Roger Federer	801
Novak Djokovic	658
Ti Chen	609
Andre Agassi	608
Yen-Hsun Lu	562

Figura 38 - Resultado 4ª query;

5. Conclusão

Durante o trabalho, surgiram desafios que obrigaram a uma reavaliação da abordagem inicial. Numa primeira fase, considerou-se realizar todo o processo de limpeza e transformação de dados apenas em SQL. No entanto, rapidamente se constataram as limitações desta abordagem, nomeadamente a dificuldade e a lentidão na execução de consultas de validação de dados complexas.

Esta constatação levou a uma alteração estratégica para uma metodologia em duas fases:

- Exploração e Limpeza em MongoDB: Utilizámos o MongoDB para a exploração, validação e limpeza inicial dos dados. Esta abordagem não relacional revelou-se mais ágil para lidar com a estrutura dos documentos JSON, permitindo identificar e resolver eficientemente inconsistências (como jogadores com o mesmo nome ou torneios com dados conflitantes) e, crucialmente, para agregar e tratar jogos duplicados, resultando na coleção `unique_matches_collection`.
- Modelo Relacional em SQL: Após esta preparação, os dados já estruturados foram exportados para CSV e importados para um modelo relacional em SQL, onde as relações entre jogadores, torneios e países puderam ser formalmente estabelecidas.

Um dos maiores obstáculos do projeto surgiu na etapa de limpeza de dados em SQL, particularmente na correspondência das localizações (campos `Born` e `Location`) com os respetivos países.

A tentativa de melhorar esta correspondência, através da criação da tabela `country_location_match` e do cruzamento com fontes de dados externas (como o ficheiro `worldcities_small`), revelou-se complexa. Foram detetados potenciais erros de correspondência (ex: cidades com o mesmo nome em países diferentes) e ligeiras diferenças nos resultados consoante os métodos de aplicação, via a utilização de comandos SQL ou via a importação da tabela `worldcities_small` em dump.

Este desafio expôs ainda a necessidade crítica de garantir uma encodificação de caracteres (encoding) correta e consistente em todo o processo, especialmente ao lidar com um dataset que inclui nomes e locais com acentos e outros caracteres de interpretação complexa.

Apesar das dificuldades encontradas, as soluções implementadas, combinando a flexibilidade do MongoDB para a exploração inicial e a robustez do SQL para a análise relacional, garantiram a construção de um modelo relacional limpo e funcional. Este modelo permitiu a realização de análises detalhadas sobre jogadores, torneios e países, respondendo com sucesso às queries-objetivo definidas.